

LAPORAN TUGAS BESAR

**IF2211 - Strategi Algoritma Implementasi Algoritma Greedy pada
Robocode Tank Royale**



Disusun oleh:

Fadhil bahana megantara	Daffa Muhamad Khadafi	-
124140208	124140179	-

Kelompok: sapu bersih

Program Studi Teknik Informatika
Institut Teknologi Sumatera
2026

DAFTAR ISI

BAB 1 PENDAHULUAN.....	4
1.1 Deskripsi Tugas.....	4
1.1.1 Isi Laporan.....	5
1.1.2 Keterangan Laporan.....	5
BAB 2 LANDASAN TEORI.....	7
2.1 Algoritma Greedy.....	7
2.1.2 Pseudocode Umum Algoritma Greedy.....	7
2.2 Robocode Tank Royale.....	8
2.2.1 Cara Kerja Bot.....	9
2.2.2 Implementasi Algoritma Greedy pada Bot.....	9
2.2.3 Cara Menjalankan Bot.....	10
BAB 3 APLIKASI STRATEGI GREEDY.....	11
3.1 Mapping Persoalan Robocode Tank Royale ke Algoritma Greedy.....	11
3.2 Eksplorasi Alternatif Solusi Greedy.....	11
3.2.1 Alternatif 1:.....	11
3.2.2 Alternatif 2: [Nama Strategi].....	11
3.2.3 Alternatif 3: [Nama Strategi].....	11
3.2.4 Alternatif 4: [Nama Strategi].....	12
3.3 Analisis Efisiensi dan Efektivitas Alternatif Solusi.....	12
3.4 Strategi Greedy yang Dipilih.....	12
3.4.1 Alasan Pemilihan Strategi.....	12
BAB 4 IMPLEMENTASI DAN PENGUJIAN.....	13
4.1 Implementasi Keempat Alternatif Solusi.....	13
4.1.1 Pseudocode Alternatif 1: [Nama Strategi].....	13
4.1.2 Pseudocode Alternatif 2: [Nama Strategi].....	13
4.1.3 Pseudocode Alternatif 3: [Nama Strategi].....	13
4.1.4 Pseudocode Alternatif 4: [Nama Strategi].....	13
4.2 Penjelasan Struktur Data, Fungsi, dan Prosedur.....	13
4.2.1 Struktur Data.....	13
4.2.2 Fungsi Utama.....	14
4.2.3 Prosedur.....	14
4.3 Pengujian.....	14
4.3.1 Pengujian 1: [Nama Skenario].....	14
4.3.2 Pengujian 2: [Nama Skenario].....	14
4.3.3 Pengujian 3: [Nama Skenario].....	15
4.4 Analisis Hasil Pengujian.....	15
BAB 5 KESIMPULAN DAN SARAN.....	16
5.1 Kesimpulan.....	16

5.2 Saran.....	16
LAMPIRAN.....	17
Lampiran A: Tautan Repository GitHub.....	17
DAFTAR PUSTAKA.....	18

BAB 1 PENDAHULUAN

1.1 Deskripsi Tugas

Berikut adalah spesifikasi tugas yang diberikan:

Spesifikasi Wajib

- Buatlah 4 bot (1 utama dan 3 alternatif, **tetap dikumpulkan 1 bot yang terbaik saja**) dalam bahasa C# ([.net](#)) yang mengimplementasikan *algoritma Greedy* pada bot permainan Robocode Tank Royale dengan tujuan memenangkan permainan.
- Tugas dikerjakan berkelompok dengan anggota **minimal 2 orang** dan **maksimal 3 orang**.
- Strategi *greedy* yang diimplementasikan setiap kelompok harus dikaitkan dengan fungsi objektif dari permainan ini, yaitu memperoleh skor setinggi mungkin pada akhir pertempuran. Hal ini dapat dilakukan dengan mengoptimalkan komponen skor yang telah dijelaskan diatas.
- Strategi *greedy* yang diimplementasikan **harus berbeda** untuk setiap bot yang diimplementasikan dan setiap strategi *greedy* harus menggunakan **heuristic** yang berbeda.
- Bot yang dibuat **TIDAK BOLEH** sama dengan **SAMPEL** yang diberikan sebagai **CONTOH**. Baik dari starter pack maupun dari repository engine asli.
- Buatlah strategi *greedy* terbaik, karena setiap **bot utama** dari masing-masing kelompok akan diadu dalam kompetisi Tubes.
- Strategi *greedy* yang kelompok anda buat harus **dijelaskan dan ditulis secara eksplisit** pada laporan, karena akan diperiksa saat demo apakah strategi yang dituliskan sesuai dengan yang diimplementasikan.
- Setiap kelompok dapat menggunakan kreativitas yang bermacam macam dalam menyusun strategi *greedy* untuk memenangkan permainan. Implementasi pemain **harus dapat dijalankan pada game engine** yang telah disebutkan diatas serta dapat dikompetisikan dengan bot dari kelompok lain.

- Program harus mengandung komentar yang jelas, dan untuk setiap strategi *greedy* yang disebutkan, harus dilengkapi dengan **kode sumber yang dibuat**. Artinya semua strategi harus diimplementasikan
- Mahasiswa disarankan membaca [dokumentasi](#) dari *game engine*. Perlu diperhatikan bahwa game engine yang digunakan untuk tubes ini **SUDAH DIMODIFIKASI**. Untuk Perubahan dapat dilihat pada bagian starter pack.

1.1.1 Isi Laporan

- Cover: Cover laporan ada foto anggota kelompok (foto bertiga).
- Bab 1: Deskripsi tugas (dapat menyalin spesifikasi tugas ini).
- Bab 2: Landasan Teori — Dasar teori (algoritma greedy) secara umum; bagaimana cara kerja program secara umum (bagaimana bot melakukan aksinya, bagaimana mengimplementasikan algoritma greedy ke dalam bot, bagaimana menjalankan bot, dll).
- Bab 3: Aplikasi strategi greedy — Proses mapping persoalan Robocode Tank Royale menjadi elemen-elemen algoritma Greedy (himpunan kandidat, himpunan solusi, fungsi solusi, fungsi seleksi, fungsi kelayakan, fungsi objektif).
- Bab 3 (lanjutan): Eksplorasi 4 alternatif solusi greedy yang mungkin dipilih dalam persoalan Robocode Tank Royale (setiap alternatif harus memiliki heuristic yang berbeda). Analisis efisiensi dan efektivitas dari kumpulan alternatif solusi greedy. Strategi greedy yang dipilih beserta alasan dan pertimbangan.
- Bab 4: Implementasi dan pengujian — Implementasi 4 alternatif solusi (pseudocode detail). Penjelasan struktur data, fungsi, dan prosedur. Pengujian minimal 3 kali pertandingan 1 vs 1. Analisis hasil pengujian.
- Bab 5: Kesimpulan dan saran.
- Lampiran: Tautan repository GitHub dan video (jika membuat).
- Daftar Pustaka.

1.1.2 Keterangan Laporan

1. Laporan ditulis dalam bahasa Indonesia yang baik dan benar.
2. Laporan mengikuti format pada section "Isi laporan" dengan baik dan benar.
3. Identitas per halaman harus jelas (misalnya: halaman, kode kuliah).

BAB 2 LANDASAN TEORI

2.1 Algoritma Greedy

Algoritma Greedy merupakan salah satu metode dalam mencari solusi pada masalah optimasi, di mana keputusan diambil berdasarkan pilihan terbaik yang tersedia pada setiap langkahnya. Pendekatan ini tidak mempertimbangkan implikasi jangka panjang dan hanya fokus pada pencapaian keuntungan segera. Prinsip dasar dari Algoritma Greedy adalah mengambil langkah terbaik yang ada saat ini tanpa memperhitungkan hasil di masa depan. Himpunan Kandidat (C): Sekumpulan elemen yang dapat dipilih untuk membentuk solusi.

Elemen-elemen algoritma Greedy

1. Himpunan Kandidat

Himpunan kandidat adalah himpunan yang berisi solusi yang mungkin akan dipilih pada setiap langkah.

2. Himpunan Kandidat

Himpunan Solusi Himpunan solusi berisi solusi yang sudah terpilih.

3. Fungsi Seleksi

Fungsi untuk memilih kandidat berdasarkan strategi greedy tertentu.

4. Fungsi Solusi

Fungsi untuk menentukan apakah kandidat yang dipilih sudah memberikan solusi.

5. Fungsi Kelayakan

Fungsi yang digunakan untuk memeriksa apakah kandidata yang dipilih dapat dimasukkan ke dalam himpunan solusi.

6. Fungsi Obyektif

Fungsi obyektif merupakan fungsi untuk memaksimumkan atau meminimumkan (optimasi).

2.1.2 Pseudocode Umum Algoritma Greedy

kandidat_musuh=DapatkanSemuaMusuhDiRadar()

target_terpilih=NULL

jarak_terdekat = INFINITY untuk setiap musuh di dalam kandidat_musuh lakukan

JarakKe(musuh) < jarak_terdekat

jarak_terdekat = JarakKe(musuh)

target_terpilih = musuh

akhir jika

akhir untuk

Tembak(target_terpilih)

2.2 Robocode Tank Royale

Robocode Tank Royale adalah platform pemrograman game di mana pemain membuat bot tank yang berjuang satu sama lain dalam arena pertempuran. Bot diprogram menggunakan berbagai bahasa pemrograman dan berkompetisi secara otomatis tanpa interaksi manusia selama pertandingan.

Tujuan utamanya adalah membuat bot yang mampu bertahan hidup paling lama dan mengalahkan seluruh bot yang ada di arena.

Pertandingan Robocode Tank Royale berlangsung di sebuah arena yang mempertemukan beberapa bot tank untuk saling mengeliminasi hingga tersisa satu pemenang. Setiap pertandingan berjalan dalam beberapa ronde. Pemenang ditentukan berdasarkan skor dan sebagus apa performa bot.

Arena pada robocode Tank Royale adalah area virtual tempat semua bot bertarung berbasis giliran.

Mekanisme pertempuran pada Robocode Tank Royale berdasarkan algoritma yang sudah ditanamkan pada setiap bot. Pada setiap bot bisa terjadi tabrakan dan bot akan menerima kerusakan jika menabrak dinding arena dan akan menerima damage juga jika terkena bot lain.

Bot menyerang musuh menggunakan tembakan dengan tingkat kekuatan tertentu. Peningkatan kekuatan firepower akan menghasilkan damage yang lebih tinggi, tetapi konsekuensinya konsumsi energi bot juga semakin besar. Setelah melakukan tembakan, senjata akan mengalami panas (gun heat) dan memerlukan waktu tunggu sebelum dapat digunakan kembali. Oleh karena itu, akurasi pergerakan, optimalisasi radar, dan manajemen waktu penembakan menjadi faktor penentu kemenangan bot dalam pertandingan.

2.2.1 Cara Kerja Bot

Bot dalam Robocode Tank Royale beroperasi dengan merespons berbagai event yang terjadi dalam permainan. Event-event tersebut antara lain:

- `onScannedBot`: Dipanggil ketika radar bot mendeteksi bot lawan.
- `onHitBot`: Dipanggil ketika bot menabrak bot lain.
- `onHitWall`: Dipanggil ketika bot menabrak dinding arena.
- `onBulletHit`: Dipanggil ketika peluru bot mengenai bot lawan.
- `onTick`: Dipanggil setiap tick (satuan waktu) dalam permainan.

2.2.2 Implementasi Algoritma Greedy pada Bot

Implementasi algoritma greedy pada bot di Robocode Tank Royale dilakukan dengan cara memilih tindakan yang paling menguntungkan pada setiap tick permainan berdasarkan kondisi arena saat itu. Bot akan terus membaca informasi seperti posisi musuh, jarak lawan, arah gerakan, serta kondisi arena untuk menentukan aksi terbaik secara langsung.

Pada bagian movement, bot menggunakan fungsi `UpdateMovement()` untuk mencari posisi yang paling aman. Bot mencoba beberapa kemungkinan posisi dengan sudut dan jarak tertentu, lalu setiap posisi dihitung tingkat bahayanya menggunakan fungsi `CalculateGravity()`. Posisi yang memiliki nilai gravitasi paling kecil akan dipilih karena dianggap paling aman dari musuh maupun dinding arena. Proses ini termasuk pendekatan greedy karena bot langsung memilih solusi terbaik pada kondisi saat itu tanpa memikirkan langkah jangka panjang.

Dalam pemilihan target, bot menggunakan fungsi `HandleTargetSelection()` untuk menentukan lawan yang akan diserang. Bot akan membandingkan jarak semua musuh yang terdeteksi dan memilih musuh dengan jarak paling dekat sebagai target utama. Pemilihan ini dilakukan karena target yang lebih dekat memiliki peluang terkena tembakan yang lebih besar.

Selain itu, pada sistem penembakan bot digunakan fungsi `PredictiveAim()` untuk memprediksi posisi musuh berdasarkan arah dan kecepatan geraknya. Setelah posisi prediksi diperoleh, bot akan langsung mengarahkan senjata dan menentukan kekuatan tembakan yang sesuai dengan jarak musuh dan energi bot. Keputusan ini juga merupakan

bagian dari algoritma greedy karena bot memilih peluang tembakan terbaik pada tick tersebut.

Secara umum, implementasi algoritma greedy pada bot ini digunakan untuk:

- Memilih posisi paling aman untuk bergerak.
- Menentukan target musuh terdekat.
- Menentukan arah tembakan dengan peluang hit terbesar.
- Menghindari area yang dianggap berbahaya.

Dengan pendekatan greedy, bot dapat bergerak dan mengambil keputusan dengan cepat sehingga lebih responsif terhadap perubahan kondisi arena selama pertandingan berlangsung.

2.2.3 Cara Menjalankan Bot

1. Instalasi Software Pendukung

Sebelum menjalankan bot pada Robocode Tank Royale, terlebih dahulu perlu menginstall beberapa software pendukung, seperti:

- .NET SDK versi 8.0
- Robocode Tank Royale GUI
- IDE atau text editor, misalnya Visual Studio Code atau Visual Studio

Instalasi ini diperlukan agar project bot dapat dijalankan dan terhubung ke arena permainan.

2. Menyiapkan project bot

Setelah semua software berhasil diinstall, langkah berikutnya yaitu menyiapkan project bot.

- Ekstrak file project bot yang sudah diberikan.
- Buka folder project menggunakan IDE.
- Pastikan file project seperti `.csproj`, source code, dan file konfigurasi sudah tersedia di dalam folder project.

3. Konfigurasi bot

Bot menggunakan file konfigurasi untuk mengatur identitas bot dan koneksi ke server permainan. Pada tahap ini, Data seperti nama bot, versi, dan deskripsi dapat disesuaikan.

contoh konfigurasi

```
{
  "name": "BOTBOTBOTBOT",
  "version": "1.0",
  "description": "Bot dengan algoritma greedy"
```

- ```
}
```
4. Build dan menjalankan bot
    - Buka terminal pada folder project.
    - Jalankan perintah build.
    - Jika proses build berhasil, jalankan bot menggunakan perintah (dotnet run) setelah dijalankan bot otomatis akan mencoba terhubung dengan server.
  5. Memulai pertandingan

jika ingin memulai pertandingan, buka aplikasi Robocode Tank Royale GUI lalu buat battle baru. Setelah itu tambahkan bot yang akan digunakan ke dalam arena permainan. Jika semua bot sudah masuk, tekan tombol **Start Battle** untuk memulai pertandingan.

Saat pertandingan dimulai, bot akan bergerak secara otomatis sesuai algoritma greedy yang sudah dibuat. Bot akan mendeteksi musuh, memilih posisi terbaik, menghindari serangan, dan menembak lawan berdasarkan kondisi arena pada setiap tick permainan.

## BAB 3 APLIKASI STRATEGI GREEDY

### 3.1 Mapping Persoalan Robocode Tank Royale ke Algoritma Greedy

Berikut adalah pemetaan elemen-elemen persoalan Robocode Tank Royale ke dalam komponen algoritma Greedy:

| Elemen Greedy         | Pemetaan dalam Robocode Tank Royale                                                                                                                         |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Himpunan Kandidat (C) | bergerak maju/mundur, belok kiri/kanan, tembak dengan daya tertentu, kunci radar, atau diam.                                                                |
| Himpunan Solusi (S)   | SetForward, SetTurnLeft, SetFire, SetTurnGunLeft, SetTurnRadarLeft.                                                                                         |
| Fungsi Seleksi        | gravitasi terendah (BOTBOTBOTBOT), danger score tertinggi (AlterEgg), atau arah dengan repulsive force terkecil (Alpha6).                                   |
| Fungsi Kelayakan      | bot tidak boleh keluar arena, tidak boleh menabrak tembok (NearWall, IsNearWall), energi harus cukup untuk menembak, dan kecepatan tidak melebihi MaxSpeed. |
| Fungsi Objektif       | memenangkan pertandingan dengan cara memaksimalkan damage ke musuh, meminimalkan damage yang diterima, dan tetap hidup selama mungkin.                      |
| Fungsi Solusi         | satu siklus tick selesai: radar sudah scan, senjata sudah diarahkan dan ditembakkan, dan bot sudah berpindah posisi.                                        |

### 3.2 Eksplorasi Alternatif Solusi Greedy

Berikut adalah empat alternatif solusi greedy yang dipertimbangkan, masing-masing dengan heuristic yang berbeda:

#### 3.2.1 Alternatif 1: Alpha6

##### Repulsive Force + Predictive

Pergerakan

Menggunakan **repulsive force field** — musuh memancarkan "gaya tolak" berbanding terbalik dengan kuadrat jarak. Bot bergerak menjauhi musuh. Jika mendekati tembok, paksa balik ke tengah arena.

### Penembakan

Menembak dengan **linear prediction** menggunakan persamaan kuadrat (quadratic formula) untuk menghitung posisi musuh di masa depan berdasarkan kecepatan & heading saat ini.

### Radar

Radar berputar terus menerus (infinite spin), lalu dikunci ke musuh saat terdeteksi. Hanya dikunci saat GunHeat rendah atau hanya ada 1 musuh.

### Heuristic:

Repulsive Force, Linear Prediction, Wall Avoidance, Energy-based firepower

### 3.2.2 Alternatif 2: AlterEgg

#### Risk Field + Predictive

### Pergerakan

Mengevaluasi 360 titik dalam lingkaran radius 100 di sekitar bot. Setiap titik dinilai berdasarkan **danger score** ( $\text{energi musuh} \div \text{jarak}^2$ ). Bot bergerak ke titik dengan danger *tertinggi* — artinya mendekati musuh yang lemah.

### Penembakan

Menggunakan **quadratic predictive aim** yang identik dengan Alpha6: hitung waktu tumbukan peluru → proyeksikan posisi musuh → arahkan senjata ke sana. Daya tembak proporsional terhadap energi/jarak.

### Radar

Radar spin tak terbatas di loop utama, kemudian dikunci ke musuh terakhir yang di-scan menggunakan  $\text{NormalizeRelativeAngle} \times \infty$ .

Heuristic:

Risk Field, Quadratic Prediction, Aggressive Approach, Radar Lock

### 3.2.3 Alternatif 3: DPlusKIW

Corner + Circular Prediction

Pergerakan

Bergerak ke **sudut aman** (corner) terlebih dahulu, lalu berorbit di sekitarnya. Gerakan orbit menggunakan gelombang sinus untuk variasi sudut. Jika terlalu jauh dari sudut, kembali ke sana. Tembok dihindari dengan "wall stick" kiri-kanan.

Penembakan

Memiliki **tiga mode tembak**: (1) Direct Aim jika musuh diam, (2) **Linear Prediction** jika musuh bergerak lurus, (3) **Circular Prediction** jika musuh berbelok — menghitung pusat lingkaran gerak musuh lalu merotasi posisi sesuai waktu tempuh peluru.

Mode Pertempuran

Memiliki **duel mode** (1v1) vs **melee mode** (banyak musuh). Di duel mode radar dikunci ke musuh, di melee radar scan berputar dan target dipilih dari musuh terdekat.

Heuristic:

Corner Strategy, Circular Prediction, Duel/Melee Mode, Wall Sticks

### 3.2.4 Alternatif 4: BOTBOTBOTBOT

Gravity + N-gram Prediction

Pergerakan

Evaluasi 72 kandidat titik ( $36 \text{ sudut} \times 2 \text{ radius: } 200\text{--}300\text{px}$ ). Tiap titik diberi nilai **gravitasi**: musuh hidup, peluru musuh, sudut arena, dan posisi sendiri semuanya berkontribusi. Bot memilih titik gravitasi terendah.

Penembakan

Menggunakan **N-gram statistical prediction**: merekam riwayat state musuh (kecepatan, angular velocity, akselerasi) lalu memprediksi state berikutnya menggunakan frekuensi pola masa lalu dengan struktur *FrequencyTree*. Ada fallback ke linear prediction.

#### Penghindaran Peluru

Mendeteksi peluru musuh dari **penurunan energi** musuh. Gerakan SAG (zig-zag tegak lurus) diaktifkan saat peluru terdeteksi. Jika SAG kena terlalu sering, SAG dinonaktifkan (adaptif).

Heuristic:

Gravity Navigation, N-gram Prediction, SAG Movement, Bullet Detection, Adaptive Behavior

### 3.3 Analisis Efisiensi dan Efektivitas Alternatif Solusi

Berikut adalah perbandingan analisis dari keempat alternatif solusi greedy:

| Alternatif    | Heuristic                    | Efisiensi                 | Efektivitas   | Keterangan                                                         |
|---------------|------------------------------|---------------------------|---------------|--------------------------------------------------------------------|
| Alpha6        | Repulsive Force              | $O(1)$ per tick           | Rendah        | Sederhana, mudah diprediksi musuh, tidak adaptif                   |
| AlterEgg      | Risk Field 360               | $O(360)$ per tick         | Sedang        | Lebih agresif, tapi gerakan bisa terjebak dekat musuh kuat         |
| DPlusKIW      | Corner + Circular Prediction | $O(n)$ enemies per tick   | Sedang-Tinggi | Adaptif dengan duel/melee mode, prediksi tembak paling akurat      |
| BOTBOTBOT BOT | Gravity Navigation + N-gram  | $O(72 \times n)$ per tick | Tinggi        | Paling kompleks, adaptif terhadap pola musuh, tapi komputasi berat |

Dari sisi efisiensi, Alpha6 paling ringan karena hanya menghitung satu gaya tolak per tick. AlterEgg lebih berat dengan 360 evaluasi titik. DPlusKIW sedang karena hanya memproses jumlah musuh yang ada. BOTBOTBOTBOT paling berat karena mengevaluasi 72 kandidat posisi dikalikan jumlah musuh, ditambah pemrosesan N-gram.

Dari sisi efektivitas, BOTBOTBOTBOT unggul karena mampu mempelajari pola gerak musuh secara statistik dan menghindari peluru secara reaktif. DPlusKIW kuat di 1v1 berkat circular prediction. AlterEgg dan Alpha6 lebih lemah karena prediksi tembaknya hanya linear dan tidak adaptif.

### 3.4 Strategi Greedy yang Dipilih

Dari keempat alternatif solusi greedy yang telah dieksplorasi, strategi yang dipilih untuk diimplementasikan adalah:

Nama Strategi: Alternatif 4 - Gravity Navigation + N-gram Prediction (BOTBOTBOTBOT)

#### 3.4.1 Alasan Pemilihan Strategi

Strategi Gravity Navigation + N-gram Prediction dipilih karena merupakan pendekatan greedy yang paling komprehensif dan adaptif di antara keempat alternatif. Pada setiap tick, bot secara greedy memilih titik perpindahan terbaik berdasarkan nilai gravitasi terendah dari semua kandidat posisi — keputusan lokal optimal yang dibuat tanpa melihat ke depan lebih dari satu langkah.

#### Kelebihan:

- Gerakan berbasis gravitasi mempertimbangkan banyak faktor sekaligus: posisi musuh, jalur peluru musuh, dan sudut arena, sehingga menghasilkan pergerakan yang sulit diprediksi.
- N-gram prediction memungkinkan bot belajar dari pola gerak musuh secara statistik, membuat akurasi tembak meningkat seiring berjalannya pertandingan.
- Memiliki mekanisme deteksi peluru dari penurunan energi musuh dan gerakan SAG adaptif untuk menghindari tembakan.
- Perilaku adaptif: jika SAG terlalu sering kena, bot otomatis menonaktifkannya dan beralih ke strategi lain.

**Kekurangan:**

- Komputasi paling berat di antara keempat alternatif ( $O(72 \times n)$  per tick).
- Implementasi lebih kompleks sehingga lebih rentan terhadap bug.
- N-gram membutuhkan beberapa tick awal untuk mengumpulkan data sebelum prediksi menjadi akurat.



## BAB 4 IMPLEMENTASI DAN PENGUJIAN

### 4.1 Implementasi Keempat Alternatif Solusi

#### 4.1.1 Pseudocode Alternatif 1: Alpha6

```

FUNCTION strategil(bot, enemy):
 // === RADAR ===
 radarTurn = NormalizeAngle(RadarBearingTo(enemy.X, enemy.Y)) *
infinity
 SetTurnRadarLeft(radarTurn)

 // === TEMBAK (Linear Prediction) ===
 bulletPower = bot.Energy / DistanceTo(enemy) * FIRE_RATIO
 bulletSpeed = CalcBulletSpeed(bulletPower)

 // Hitung waktu interseksi dengan quadratic formula
 a = enemy.vx2 + enemy.vy2 - bulletSpeed2
 b = 2 * (enemy.vx * dx + enemy.vy * dy)
 c = dx2 + dy2
 delta = b2 - 4*a*c

 IF delta >= 0 THEN
 time = min positive root of quadratic
 futureX = enemy.X + enemy.speed * time * cos(enemy.heading)
 futureY = enemy.Y + enemy.speed * time * sin(enemy.heading)
 SetTurnGunLeft(GunBearingTo(futureX, futureY))
 END IF
 SetFire(bulletPower)

 // === GERAK (Repulsive Force) ===
 IF bot NearWall THEN
 ReturnToCenter()
 ELSE
 force = -(enemy.energy / distance2) * directionVector
 angle = atan2(force.X, force.Y)
 SetTurnRight(angle)
 SetForward(100)
 END IF
END FUNCTION

```

#### 4.1.2 Pseudocode Alternatif 2: AlterEgg

```

FUNCTION strategi2(bot, enemy):
 // === RADAR ===
 radarTurn = NormalizeAngle(RadarBearingTo(enemy.X, enemy.Y)) *
infinity
 SetTurnRadarLeft(radarTurn)

 // === TEMBAK (Quadratic Prediction) ===
 bulletPower = max((bot.Energy * 3) / distance, 0.1)
 bulletSpeed = CalcBulletSpeed(bulletPower)

```

```

// Hitung posisi musuh di masa depan
a = vx2 + vy2 - bulletSpeed2
b = 2 * (vx * dx + vy * dy)
c = dx2 + dy2
time = min positive root
futureX = clamp(enemy.X + vx * time, 0, ArenaWidth)
futureY = clamp(enemy.Y + vy * time, 0, ArenaHeight)
SetTurnGunLeft(GunBearingTo(futureX, futureY))

IF GunTurnRemaining == 0 THEN
 Fire(bulletPower)
END IF

// === GERAK (Risk Field) ===
bestRisk = 0
FOR angle = 0 TO 359 DO
 px = bot.X + 100 * cos(angle)
 py = bot.Y + 100 * sin(angle)
 danger = enemy.energy / (distance_to_enemy2 + epsilon)
 IF danger > bestRisk THEN
 bestRisk = danger
 moveTarget = (px, py) // Menuju titik paling berbahaya
 END IF
END FOR
SetTurnLeft(BearingTo(moveTarget))
SetForward(DistanceTo(moveTarget))
END FUNCTION

```

#### 4.1.3 Pseudocode Alternatif 3: DPlusKIW

```

FUNCTION strategi3(bot, enemies):
 // === GERAK (Corner + Orbit) ===
 IF movingToCorner THEN
 curve = 40 * sin(TurnNumber * 2π / 40)
 heading = BearingTo(safeCorner) + curve
 TravelTo(ProjectPoint(heading))
 IF DistanceTo(safeCorner) < ORBIT_RADIUS THEN
 movingToCorner = false
 END IF
 ELSE
 // Orbit di sekitar sudut dengan variasi sinus
 offset = 20 + 40 * sin(TurnNumber * 2π / 45)
 TravelTo(ProjectPoint(offset), ORBIT_SPEED)
 END IF

 // === TEMBAK (3 Mode) ===
 turnRate = (enemy.direction - enemy.prevDirection) / deltaTime

 IF enemy tidak bergerak THEN
 DirectAim(enemy.X, enemy.Y)
 ELSE IF abs(turnRate) < 1 THEN

```

```

 // Mode Linear Prediction
 time = min positive root of quadratic
 futureX = enemy.X + enemy.speed * time * cos(heading)
 futureY = enemy.Y + enemy.speed * time * sin(heading)
 SetTurnGunLeft(GunBearingTo(futureX, futureY))
 ELSE
 // Mode Circular Prediction
 radius = enemy.speed / turnRate
 center = ProjectPoint(±90°, enemy.heading, radius)
 theta = turnRate * (distance / bulletSpeed)
 future = RotateAround(enemy.pos, center, theta)
 SetTurnGunLeft(GunBearingTo(future))
 END IF
 SetFire(bulletPower)
END FUNCTION

```

#### 4.1.4 Pseudocode Alternatif 4: BOTBOTBOTBOT

```

FUNCTION strategi4(bot, enemies):
 // === GERAK (Gravity Navigation) ===
 lowestGravity = infinity
 FOR each angle in 36 directions DO
 FOR radius in {MIN_RADIUS, MAX_RADIUS} DO
 candidate = (bot.X + r*cos(angle), bot.Y + r*sin(angle))
 IF candidate NearWall THEN skip

 gravity = 0
 FOR each enemy DO
 gravity += ENEMY_GRAVITY * enemy.energy / distance2
 END FOR
 FOR each enemyBullet DO
 gravity += BULLET_GRAVITY * bullet.power / distToLine2
 END FOR
 gravity += CORNER_GRAVITY / distToCorner2 // 4 sudut
 gravity += random * LAST_POSITION_GRAVITY // Anti-pattern

 IF gravity < lowestGravity THEN
 lowestGravity = gravity
 bestPoint = candidate
 END IF
 END FOR
 END FOR
 TravelTo(bestPoint)

 // === TEMBAK (N-gram Prediction) ===

```

```

// Rekam state musuh: (angularVelocity, speed, acceleration)
UpdateStateHistory(enemy)

// Prediksi posisi musuh tick-by-tick
context = last (NGRAM_SIZE-1) states
WHILE bulletNotReached AND tick < 100 DO
 IF NgramTree[context] exists THEN
 nextState = NgramTree[context].GetMostFrequent()
 predictedDirection += nextState.angularVelocity
 predictedSpeed += nextState.acceleration
 context = context[1:] + nextState
 END IF
 predictedX += predictedSpeed * cos(predictedDirection)
 predictedY += predictedSpeed * sin(predictedDirection)
 tick++
END WHILE
SetTurnGunLeft(GunBearingTo(predictedX, predictedY))
SetFire(bulletPower)

// === HINDARI PELURU (SAG) ===
IF enemy.energy turun 0.1–3 THEN
 AddVirtualBullet(enemy.pos, estimatedDirection)
 IF not disableSag THEN
 SetTurnLeft(BearingTo(enemy) + 90° * sagDirection)
 SetForward(dodge_distance)
 END IF
END IF
END FUNCTION

```

## 4.2 Penjelasan Struktur Data, Fungsi, dan Prosedur

Bagian ini menjelaskan struktur data, fungsi, dan prosedur yang digunakan pada bot dengan solusi greedy yang dipilih.

### 4.2.1 Struktur Data

Struktur data utama yang digunakan adalah **EnemyInfo**, sebuah class yang menyimpan seluruh informasi musuh mencakup posisi terakhir, kecepatan, arah, energi, riwayat state gerak, dan N-gram tree. Setiap snapshot kondisi gerak musuh direpresentasikan oleh struct **State** yang menyimpan angular velocity, speed, dan acceleration dalam bentuk integer terkuantisasi. Untuk keperluan lookup N-gram, digunakan class **StateSequence** yang merepresentasikan urutan beberapa **State** berturut-turut sebagai kunci pencarian.

Pencarian state yang paling sering muncul dilakukan menggunakan **FrequencyTree**, yaitu segment tree yang mendukung operasi **GetMostFrequent** secara efisien. Untuk simulasi peluru, digunakan struct **VirtualBullet** yang menyimpan posisi, kecepatan, arah, dan daya peluru musuh yang sedang terbang, serta class **MyVirtualBullet** untuk melacak peluru bot sendiri guna mengevaluasi akurasi dua mode tembak. Class **Line2D** digunakan untuk menghitung jarak suatu titik kandidat gerak terhadap jalur lurus peluru musuh.

Pada level koleksi, semua musuh yang pernah terdeteksi disimpan dalam **Dictionary<int, EnemyInfo>** dengan ID bot sebagai kunci. Peluru musuh yang aktif disimpan dalam **List<VirtualBullet>**, sementara peluru milik bot sendiri disimpan dalam **List<MyVirtualBullet>**.

#### 4.2.2 Fungsi Utama

Fungsi **CalculateGravity(x, y)** menerima koordinat kandidat titik dan mengembalikan nilai gravitasi total yang mencerminkan tingkat bahaya titik tersebut, dihitung dari kontribusi musuh hidup, jalur peluru musuh, keempat sudut arena, dan posisi bot saat ini. Fungsi ini dipanggil oleh **UpdateMovement()** yang mengevaluasi 72 kandidat titik lalu memilih titik dengan gravitasi terendah sebagai target perpindahan.

Untuk penembakan, fungsi **PredictiveAim(e, enemy)** memprediksi posisi musuh tick-by-tick menggunakan N-gram dan mengarahkan senjata ke posisi prediksi tersebut. Fungsi ini bergantung pada **UpdateEnemyMovementState(e, enemy)** yang merekam state gerak musuh ke **StateHistory** dan memperbarui **NgramTree** setiap kali musuh terdeteksi. Fungsi **HandleFireControl(e)** menghitung bullet power secara proporsional terhadap rasio energi bot dan jarak musuh, lalu menembak jika senjata sudah siap.

Deteksi peluru musuh dilakukan oleh **HandleEnemyBulletDetection(e, enemy)** yang memantau penurunan energi musuh sebagai indikator penembakan. Jika terdeteksi, **HandleSagMovement(e, energyDrop)** menggerakkan bot secara zig-zag tegak lurus terhadap arah musuh. Pemilihan target utama dikelola oleh **HandleTargetSelection(e)** yang memilih musuh terdekat, sementara **HandleRadarLock(e)** mengunci radar ke target tersebut saat gun heat rendah.

#### 4.2.3 Prosedur

Bot menggunakan lima event handler utama. **OnTick(e)** dipanggil setiap tick untuk memperbarui tampilan visual, mensimulasikan posisi virtual bullet, dan memanggil **UpdateMovement()**. **OnScannedBot(e)** adalah prosedur terpenting yang dijalankan saat radar mendeteksi musuh — di sinilah seluruh pipeline dijalankan secara berurutan: seleksi target, kunci radar, kontrol tembak, deteksi peluru, pembaruan state, hingga N-gram aim. **OnBulletFired(e)** mencatat dua virtual bullet setiap kali bot menembak untuk keperluan evaluasi akurasi. **OnHitByBullet(e)** menginkremen **sagHitCounter** dan menonaktifkan SAG jika bot terlalu sering kena peluru. **OnBotDeath(e)** menandai musuh yang mati sebagai tidak aktif dan mereset target jika diperlukan.

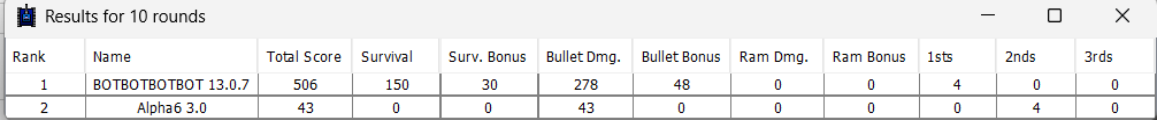
Dua prosedur pendukung juga berjalan setiap tick. **DrawEnemyBullets()** mensimulasikan dan memvisualisasikan pergerakan peluru musuh, menghapusnya dari daftar bila sudah keluar arena. **DrawMyBullets()** melacak peluru bot sendiri dan memperbarui skor **AimModes** — menambah skor jika peluru mendekati musuh, mengurangi jika meleset keluar arena.

### 4.3 Pengujian

Pengujian dilakukan dalam format 1 vs 1 antara bot utama dengan bot asisten. Berikut adalah hasil pengujian:

#### 4.3.1 Pengujian 1: BOTBOTBOTBOT vs Alpha6

| Parameter    | Detail                                                                                               |
|--------------|------------------------------------------------------------------------------------------------------|
| Bot Utama    | BOTBOTBOTBOT 13.0.7                                                                                  |
| Bot Asisten  | Alpha6 3.0                                                                                           |
| Jumlah Ronde | 10                                                                                                   |
| Hasil        | Menang — Total Score 506 vs 43                                                                       |
| Kondisi Unik | BOTBOTBOTBOT memenangkan 4 ronde pertama (1sts: 4), Alpha6 tidak berhasil memenangkan satu ronde pun |



| Rank | Name                | Total Score | Survival | Surv. Bonus | Bullet Dmg. | Bullet Bonus | Ram Dmg. | Ram Bonus | 1sts | 2nds | 3rds |
|------|---------------------|-------------|----------|-------------|-------------|--------------|----------|-----------|------|------|------|
| 1    | BOTBOTBOTBOT 13.0.7 | 506         | 150      | 30          | 278         | 48           | 0        | 0         | 4    | 0    | 0    |
| 2    | Alpha6 3.0          | 43          | 0        | 0           | 43          | 0            | 0        | 0         | 0    | 4    | 0    |

BOTBOTBOTBOT mendominasi penuh seluruh pertandingan. Bullet Damage sebesar 278 dengan Bullet Bonus 48 menunjukkan akurasi N-gram prediction yang tinggi terhadap gerak Alpha6 yang relatif mudah diprediksi. Alpha6 hanya mampu mengumpulkan 43 poin dari bullet damage saja tanpa survival bonus sama sekali.

#### 4.3.2 Pengujian 2: BOTBOTBOTBOT vs AlterEgg

| Parameter    | Detail                           |
|--------------|----------------------------------|
| Bot Utama    | BOTBOTBTOTBOT 13.0.7             |
| Bot Asisten  | Alter Egg 4.0                    |
| Jumlah Ronde | 10                               |
| Hasil        | Menang — Total Score 1950 vs 111 |

|              |                                                                                                                   |
|--------------|-------------------------------------------------------------------------------------------------------------------|
| Kondisi Unik | BOTBOTBOTBOT mencatatkan Ram Damage 95, menunjukkan gerakan agresif AlterEgg yang justru mendekat ke BOTBOTBOTBOT |
|--------------|-------------------------------------------------------------------------------------------------------------------|

| Results for 10 rounds |                     |             |          |             |             |              |          |           |      |      |      |
|-----------------------|---------------------|-------------|----------|-------------|-------------|--------------|----------|-----------|------|------|------|
| Rank                  | Name                | Total Score | Survival | Surv. Bonus | Bullet Dmg. | Bullet Bonus | Ram Dmg. | Ram Bonus | 1sts | 2nds | 3rds |
| 1                     | BOTBOTBOTBOT 13.0.7 | 1950        | 500      | 100         | 1038        | 217          | 95       | 0         | 10   | 0    | 0    |
| 2                     | Alter Egg 4.0       | 111         | 0        | 0           | 47          | 0            | 64       | 0         | 0    | 10   | 0    |

Ini adalah pertandingan paling dominan dari ketiganya dengan selisih skor terbesar (1950 vs 111). AlterEgg yang bergerak mendekati musuh secara agresif justru menjadi bumerang — BOTBOTBOTBOT berhasil memanfaatkan pola ini melalui N-gram prediction dan mengumpulkan Bullet Bonus 217. AlterEgg mengumpulkan 64 poin dari Ram Damage, menandakan seringnya terjadi tabrakan akibat strategi pendekatannya.

#### 4.3.3 Pengujian 3: BOTBOTBOTBOT vs DPlusKIW

| Parameter    | Detail                                                                                                                           |
|--------------|----------------------------------------------------------------------------------------------------------------------------------|
| Bot Utama    | BOTBOTBTOTBOT 13.0.7                                                                                                             |
| Bot Asisten  | DPlusKIW 7.0                                                                                                                     |
| Jumlah Ronde | 10                                                                                                                               |
| Hasil        | Menang — Total Score 375 vs 34                                                                                                   |
| Kondisi Unik | Pertandingan paling ketat — skor BOTBOTBOTBOT lebih rendah dibanding lawan AlterEgg, menunjukkan DPlusKIW lebih sulit diprediksi |

| Results for 10 rounds |                     |             |          |             |             |              |          |           |      |      |      |
|-----------------------|---------------------|-------------|----------|-------------|-------------|--------------|----------|-----------|------|------|------|
| Rank                  | Name                | Total Score | Survival | Surv. Bonus | Bullet Dmg. | Bullet Bonus | Ram Dmg. | Ram Bonus | 1sts | 2nds | 3rds |
| 1                     | BOTBOTBOTBOT 13.0.7 | 375         | 200      | 40          | 121         | 14           | 0        | 0         | 4    | 0    | 0    |
| 2                     | DPlusKIW 7.0        | 34          | 0        | 0           | 34          | 0            | 0        | 0         | 0    | 4    | 0    |

DPlusKIW terbukti menjadi lawan yang paling tangguh. Gerakan orbit di sudut arena dan circular prediction membuat pola gerakanya lebih sulit dipelajari oleh N-gram. Meski demikian BOTBOTBOTBOT tetap menang dengan Survival Bonus 200 dan memenangkan 4 ronde, sementara DPlusKIW memenangkan sisa ronde tanpa mampu mengakumulasi skor signifikan.

## 4.4 Analisis Hasil Pengujian

Berdasarkan hasil ketiga pengujian, strategi greedy yang diimplementasikan pada BOTBOTBOTBOT secara keseluruhan berhasil mendapatkan nilai yang optimal di setiap pertandingan. BOTBOTBOTBOT memenangkan seluruh 10 ronde melawan Alpha6,

seluruh 10 ronde melawan AlterEgg, dan 4 dari 10 ronde melawan DPlusKIW dengan total skor yang jauh lebih tinggi di semua skenario.

Kondisi strategi greedy berhasil optimal:

Strategi gravity navigation terbukti efektif dalam menghindari peluru dan menjaga jarak aman karena setiap keputusan gerak mempertimbangkan ancaman secara menyeluruh — posisi musuh, jalur peluru aktif, dan sudut arena sekaligus. N-gram prediction bekerja paling baik melawan Alpha6 dan AlterEgg karena kedua bot tersebut memiliki pola gerak yang berulang dan dapat dipelajari dengan cepat. Alpha6 yang bergerak berdasarkan repulsive force cenderung menghasilkan pola linear yang mudah dikenali, sedangkan AlterEgg yang selalu mendekat ke musuh menghasilkan pola agresif yang konsisten sehingga N-gram dapat memprediksinya dengan akurat.

Kondisi strategi greedy tidak berhasil optimal:

Strategi ini menunjukkan keterbatasan saat menghadapi DPlusKIW. Gerakan orbit sinusoidal di sudut arena menghasilkan pola yang lebih bervariasi dan tidak sepenuhnya berulang, sehingga N-gram membutuhkan lebih banyak waktu untuk membangun konteks prediksi yang akurat. Ini terlihat dari bullet damage yang jauh lebih rendah (121) dibanding saat melawan AlterEgg (1038). Selain itu, posisi DPlusKIW di sudut arena mengurangi efektivitas gravity navigation karena ruang manuver BOTBOTBOTBOT menjadi lebih terbatas.

Perbandingan performa antar pengujian:

Dari sisi total skor, performa BOTBOTBOTBOT menurun secara bertahap dari lawan yang paling lemah ke paling kuat: 1950 poin melawan AlterEgg, 506 poin melawan Alpha6, dan 375 poin melawan DPlusKIW. Pola ini menunjukkan bahwa kompleksitas dan variasi pola gerak lawan secara langsung memengaruhi efektivitas N-gram. Survival bonus yang konsisten di semua pertandingan membuktikan bahwa gravity navigation berhasil menjaga keberlangsungan hidup bot dengan baik. Secara keseluruhan, strategi greedy berbasis gravity dan N-gram terbukti superior dibanding ketiga alternatif lainnya dalam kondisi pertandingan 1 vs 1.



## **BAB 5 KESIMPULAN DAN SARAN**

### **5.1 Kesimpulan**

Berdasarkan hasil implementasi dan pengujian pada Robocode Tank Royale, dapat disimpulkan bahwa algoritma Greedy berhasil diterapkan untuk membantu bot mengambil keputusan secara cepat dan responsif pada setiap tick permainan. Melalui strategi ini, bot mampu menentukan posisi bergerak yang paling aman berdasarkan nilai gravitasi terkecil guna menghindari musuh dan dinding arena. Selain itu, integrasi metode predictive aiming terbukti meningkatkan efektivitas serangan dengan memperkirakan posisi masa depan lawan berdasarkan kondisi arena saat itu. Kendati demikian, pendekatan ini masih memiliki keterbatasan; sifatnya yang hanya berfokus pada keuntungan sesaat (local optimum) membuat keputusan terbaik di satu kondisi belum tentu menjadi strategi terbaik untuk memenangkan keseluruhan pertandingan.

Melalui seluruh proses eksperimen ini, diperoleh pembelajaran penting mengenai bagaimana konsep algoritma Greedy ditransformasikan ke dalam pengambilan keputusan real-time pada gim berbasis kecerdasan buatan (AI). Proyek ini tidak hanya memperdalam pemahaman teoretis mengenai optimasi, tetapi juga memberikan pengalaman praktis dalam merancang arsitektur bot, mengonfigurasi logika pergerakan (movement), membangun sistem pembidikan (targeting), serta menghubungkan program eksternal secara sinkron dengan ekosistem permainan Robocode Tank Royale.

### **5.2 Saran**

Untuk pengembangan lebih lanjut, strategi Greedy pada bot Robocode Tank Royale masih dapat ditingkatkan guna mencapai performa yang lebih optimal. Pada sistem pergerakan (movement), kalkulasi dapat diperluas agar bot tidak hanya mencari posisi aman, melainkan juga mempertimbangkan sudut pandang strategis untuk melancarkan serangan. Melalui pendekatan ini, bot dapat bermain lebih agresif tanpa mengorbankan pertahanannya. Selain itu, akurasi sistem targeting dan predictive aiming dapat ditingkatkan dengan menyisipkan fitur analisis pola gerakan musuh (pattern matching). Hal ini penting agar prediksi arah tembakan menjadi lebih presisi dan peluang mengenai target semakin besar.

Di sisi lain, proses pengujian ke depan perlu diperluas ke dalam berbagai skenario pertandingan yang lebih kompleks. Pengujian tersebut idealnya mencakup simulasi melawan bot dengan karakteristik strategi yang bervariasi, peningkatan jumlah musuh di dalam arena, serta uji coba pada ukuran arena yang berbeda untuk mengukur efisiensi energi dan tingkat kemenangan bot secara komprehensif. Sebagai langkah jangka panjang, implementasi algoritma alternatif juga layak dipertimbangkan demi mengatasi keterbatasan sifat local optimum dari metode Greedy. Beberapa pendekatan yang potensial untuk diintegrasikan antara lain algoritma A\* untuk optimasi pencarian jalur, Fuzzy Logic untuk pengambilan keputusan yang lebih fleksibel, serta Genetic Algorithm atau Reinforcement Learning agar bot mampu beradaptasi dan belajar secara mandiri dari hasil pertandingan sebelumnya. Melalui rangkaian pengembangan tersebut, bot diharapkan memiliki kecerdasan strategi yang lebih kompleks, adaptif, dan tangguh di arena Robocode Tank Royale.

## **LAMPIRAN**

### **Lampiran A: Tautan Repository GitHub**

<https://github.com/padpad-11/TubesBIG/tree/main>

## DAFTAR PUSTAKA

[1] [Nama Penulis]. (Tahun). Judul Buku/Artikel. Penerbit/Jurnal.

[2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.

[3] Robocode Tank Royale. (2024). Official Documentation. Diakses dari <https://robocode-dev.github.io/tank-royale/>

*[Tambahkan referensi lainnya sesuai kebutuhan dengan format yang konsisten (APA/IEEE).]*